

ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
МЭДЭЭЛЭЛ, ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ



F.CSM314 Программ хангамжийн зохиомж архитектур

Хандивын Систем

БАГИЙН АЖИЛ №8

Модуль: Хэрэглэгчийг бүртгэх, нэвтрүүлэх ба эрх зохицуулалт

<i>Шалгасан багш:</i>	Б.Алтантуяа
<i>Багийн дугаар:</i>	№4 (SmartStack)
<i>Гүйцэтгэсэн оюутнууд:</i>	Д.Нурсолтан /B232270039/ С.Асилан /B232270115/ Б.Дүүрэнбилэг /B222270032/

Улаанбаатар хот
2026 он

Агуулга

1	Төслийн ерөнхий танилцуулга	2
1.1	Системийн хэрэглэгчийн үүрэг (Roles)	2
2	Хөгжүүлэх модулийн тодорхойлолт, шаардлагууд	2
2.1	Модульд тавигдсан үндсэн шаардлагууд:	2
3	Системийн үйл ажиллагаа ба архитектур	3
4	Төслийн бүтэц (Folder Structure)	4
5	Ашигласан технологиуд	4
6	Баталгаажуулалтын урсгал (Authentication OTP Flow)	4
6.1	Хэрэглэгчийн Lifecycle дараалал	5
6.2	OTP Баталгаажуулалтын дүрэм (OTP Policy)	5
7	API Хандалтын цэгүүд (API Endpoints)	5
8	Аюулгүй байдлын тактикуудыг хэрхэн хэрэгжүүлсэн бэ?	6
9	Тестийн үр дүн (Test Results)	6
10	Үзүүлэн харуулах дараалал (Demo Flow)	6
11	Дүгнэлт	7

Модулийн ерөнхий мэдээлэл

Нэршил:	Donation Auth Module
Үндсэн функц:	Хэрэглэгч шинээр бүртгэх, нэвтрэх, OTP кодоор баталгаажуулах, JWT access token үүсгэх, Refresh token rotation болон role-based authorization хянах.
Ашигласан тулгуур:	Node.js, Express framework, MongoDB, Mongoose ODM, Jest
Аюулгүй байдал:	bcrypt нууцлал, OTP сэргийлэлт, Refresh token хугацаа, хязгаарлалтууд
Тест гүйцэтгэл:	1 test suite, 12 tests passed (Бүрэн шалгагдсан)

1. Төслийн ерөнхий танилцуулга

Манай баг энэхүү бие даалтын ажлын хүрээнд “Хандивын систем”-ийн аюулгүй байдал, хэрэглэгчийн бүртгэл болон хандалтын эрхийг зохицуулах ****Donation Auth Module**** backend хэсгийг бие даан хөгжүүллээ. Энэхүү модуль нь системийн хэмжээнд хамгаалагдсан өгөгдлүүд рүү хууль бусаар хандахаас сэргийлэх, хэрэглэгчийг үнэн зөв танин баталгаажуулах суурь бүрэлдэхүүн хэсэг болж байгаа юм.

Төслийн бүх эх кодыг сургуулийн шаардлагын дагуу дараах GitHub репозиторид нээлттэй байршуулсан болно:

URL: <https://github.com/AsilanAsKa/BagiinAjil-8.git>

1.1 Системийн хэрэглэгчийн үүрэг (Roles)

Хөгжүүлсэн модулийн түвшинд хэрэглэгчдийг дараах 3 үндсэн бүлэгт хувааж, эрхийг заагласан:

- **Guest Donor (Хандивлогч):** Системд бүртгүүлж, OTP баталгаажуулалт хийсний үндсэн дээр нэвтэрч, нээлттэй аянуудад хандив өгөх эрхтэй хэрэглэгч.
- **Campaign Creator (Аян үүсгэгч):** Хандивын аян шинээр үүсгэх, өөрийн үүсгэсэн аяны явцыг хянах, тайлан оруулах эрхтэй оролцогч.
- **Admin (Администратор):** Системийн бүх хэрэглэгчдийн мэдээллийг харах, аянуудыг батлах болон устгах эрх бүхий дээд хэрэглэгч.

2. Хөгжүүлэх модулийн тодорхойлолт, шаардлагууд

Энэхүү модулийг хөгжүүлэхдээ Бие даалт-2 дээр тодорхойлсон шаардлагуудыг бүрэн хангахыг зорив. Үүнд хэрэглэгчээс ирж буй өгөгдлийг шалгах, нууц үгийн нууцлалыг хангах, мөн хандалтын хугацааг зөв удирдах асуудлууд багтсан.

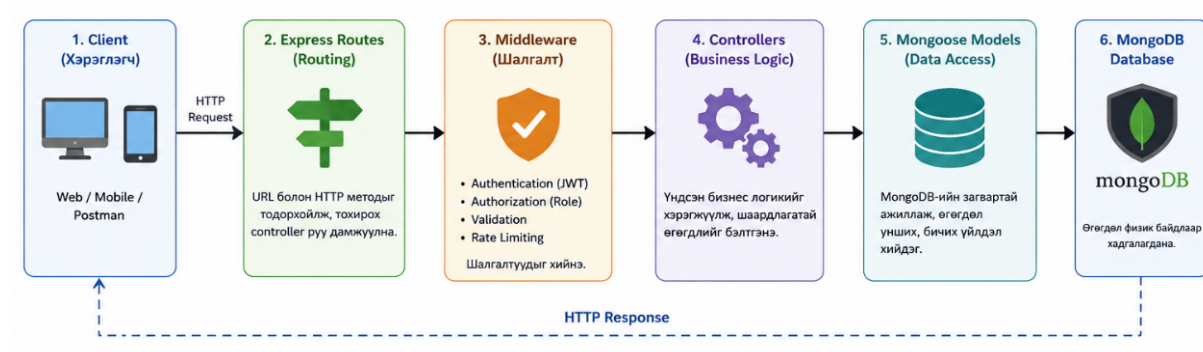
2.1 Модульд тавигдсан үндсэн шаардлагууд:

- Бүртгэлийн үед ирж буй мэдээллүүдийг (email, phone, password, role) оролтын түвшинд шалгаж, буруу өгөгдлийг буцаадаг байх.

- Хэрэглэгчийн нууц үгийг мэдээллийн санд plain-text буюу шууд уншигдах хэлбэрээр хэзээ ч хадгалахгүй байх.
- Бүртгүүлсэн даруйд хэрэглэгчийг шууд идэвхжүүлэхгүй, заавал 6 оронтой OTP кодоор баталгаажуулсны дараа идэвхтэй төлөвт шилжүүлэх.
- Системийн аюулгүй байдлыг хангах үүднээс богино хугацааны **Access Token** болон урт хугацааны **Refresh Token** ашиглан session удирдах.
- Хамгаалагдсан хандалтын цэг (API Route) бүр дээр хэрэглэгчийн role-ийг middleware ашиглан шалгадаг байх.

3. Системийн үйл ажиллагаа ба архитектур

Манай модуль нь программ хангамжийн ****Layered Architecture (Үелсэн архитектур)**** загварын дагуу хийгдсэн. Энэ нь кодын уялдаа холбоог салгаж, дараа нь өргөтгөх болон тест бичихэд маш хялбар болгож өгдөг.



Зураг 1: Системийн өгөгдлийн урсгал болон архитектурын үе давхаргууд

Зураг 1-т харуулсны дагуу хэрэглэгчээс ирсэн хүсэлт нь Middleware-ээр дамжин баталгаажигд, Controller давхаргад боловсруулагдан Mongoose моделиор дамжиж MongoDB-д хадгалагдах урсгалтай байна.

Хүснэгт 1: Архитектурын үе давхаргууд

Давхарга	Гүйцэтгэх үүрэг	Файл / Жишээ
Application	Express app-ийг ажиллуулж, глобал тохиргоонуудыг ачаалах	backend/src/app.js
Routes	API хандалтын цэгүүдийг зааж, validator-уудыг холбож өгөх	authRoutes.js, userRoutes.js
Controllers	Бүртгүүлэх, нэвтрэх, OTP шалгах зэрэг үндсэн бизнес логикийг гүйцэтгэх	authController.js
Middleware	Хэрэглэгчийн JWT token болон Role-ийг замаас нь барьж шалгах	auth.js, validate.js

Давхарга	Гүйцэтгэх үүрэг	Файл / Жишээ
Data Model	Мэдээллийн сангийн бүтцийг (User schema) тодорхойлох	User.js

4. Төслийн бүтэц (Folder Structure)

Кодын зохион байгуулалтыг дараах хавтасны бүтцийн дагуу цэгцтэй байршуулсан:

donation-auth-module	
backend/ .env.example package.json src/ app.js server.js config/ controllers/ middleware/ models/ routes/ tests/ utils/ docs/ README.md	 # Express аппликейшн тохиргоо # Сервер асах цэг # Database холболтын тохиргоо # Бизнес логик (Controller давхарга) # Эрх шалгах middleware-үүд # Схем загварууд (User, OTP) # Хандалтын замууд (Router) # Интеграци тестүүд (Jest) # Туслах функцууд (Token, OTP generation) # Төслийн баримт бичгүүд

5. Ашигласан технологиуд

Бид модулиа хөгжүүлэхдээ backend орчинд өргөн ашиглагддаг, найдвартай дараах технологиудыг сонгон авсан:

- **Node.js & Express framework:** Backend логикийг хурдан ажиллуулах, REST API үүсгэх үндсэн суурь.
- **MongoDB & Mongoose ODM:** Хэрэглэгч болон Token-ий мэдээллийг уян хатан хадгалах NoSQL өгөгдлийн сан.
- **bcrypt (v6.0.0):** Нууц үг хадгалахад ашигласан урсгал криптограф хаш сан. Аюулгүй байдлын үүднээс хамгийн сүүлийн хувилбарыг сонгосон.
- **jsonwebtoken (JWT):** Хэрэглэгч нэвтэрснийг батлах token үүсгэх, түүнийг баталгаажуулах сан.
- **express-rate-limit & Helmet:** Нэг дороос олон хандалт хийж системийг унагаах эсвэл нууц үг таахаас сэргийлэх хязгаарлагч ба HTTP хамгаалалтын header сан.
- **Jest & Supertest:** Системийн функцуудыг зөв ажиллаж байгааг автоматжуулж шалгах тестийн орчин.

6. Баталгаажуулалтын урсгал (Authentication OTP Flow)

Системд хэрэглэгч нэвтрэх урсгал болон OTP кодын зохицуулалтыг дараах алгоритмаар шийдсэн:

6.1 Хэрэглэгчийн Lifecycle дараалал

Бүртгүүлэх → OTP үүсэх → OTP шалгах → Хэрэглэгч идэвхжих → Нэвтрэх → Token авах → Хандалт хийх → Token шинэчлэх (Refresh) → Системээс гарах

6.2 OTP Баталгаажуулалтын дүрэм (OTP Policy)

Бүртгэлийг баталгаажуулахдаа brute-force буюу код таах халдлагаас сэргийлж дараах дүрмийг суулгасан:

- **Кодын урт:** 6 оронтой тоо байна.
- **Хүчинтэй хугацаа:** 2 минут (TTL). Хугацаа дуусвал уг код идэвхгүй болно.
- **Буруу оролдлого:** Хэрэв 3 удаа дараалан буруу код оруулбал тухайн OTP баталгаажуулалтыг түр цоожилно (isLocked=true).
- **Нууцлал:** Үүссэн OTP кодын raw утгыг database-д хадгалахгүй, мөн адил bcrypt ашиглан hash хийж хадгална.

7. API Хандалтын цэгүүд (API Endpoints)

Системд дараах үндсэн endpoint-уудыг үүсгэж, хандалтын түвшинг тодорхойлж өгсөн:

Хүснэгт 2: Хөгжүүлсэн API-уудын жагсаалт

Method	Endpoint	Хандах эрх	Үүрэг / Тайлбар
GET	/health	Нийтэд нээлттэй	Серверийн ажиллагааг шалгах
POST	/api/auth/register	Нийтэд нээлттэй	Шинэ хэрэглэгч бүртгэх, OTP үүсгэх
POST	/api/auth/verify-otp	Хязгаарлалттай	Оруулсан OTP-г шалгаж идэвхжүүлэх
POST	/api/auth/login	Хязгаарлалттай	Системд нэвтрэх
POST	/api/auth/refresh	Хязгаарлалттай	Refresh token-оор шинэ Access Token авах
POST	/api/auth/logout	Нийтэд нээлттэй	Системээс гарах, token устгах
GET	/api/users/me	Нэвтэрсэн хэрэглэгч	Өөрийн профайл мэдээллийг харах
GET	/api/admin/users	Зөвхөн Админ	Бүх хэрэглэгчдийн жагсаалтыг харах

Хамгаалагдсан API-уудыг дуудахдаа HTTP Request Header-ийн Authorization хэсэгт token-ийг дамжуулна:

```
Authorization: Bearer <jwt_access_token>
```

8. Аюулгүй байдлын тактикуудыг хэрхэн хэрэгжүүлсэн бэ?

Хичээлийн шаардлагын дагуу манай баг аюулгүй байдлын дараах тактикуудыг backend кодын түвшинд бодитоор хэрэгжүүлсэн:

- **Нууц үгийн хамгаалалт (Password Hashing):** Хэрэглэгчийн нууц үгийг database-д plain text-ээр огт хадгалахгүй, bcrypt-ээр hash хийнэ. Мөн Mongoose схем дээр `select: false` тохируулгыг хийсэн. Ингэснээр энгийн хайлтаар нууц үг, OTP кодын hash зэрэг эмзэг мэдээллүүд автоматаар listen хийгдэж гадагшаа алдагдахаас сэргийлэгдэнэ.
- **Refresh Token Rotation (RTR):** Хэрэглэгч шинэ access token авах хүсэлт гаргах болгонд хуучин refresh token-ийг шууд устгаж, шинэ token үүсгэж сольдог. Хэрэв хуучин (хүчингүй болсон) token-ийг ашиглан дундаас нь халдагч нэвтрэхийг оролдвол систем үүнийг халдлага гэж бүртгэж, тухайн хэрэглэгчийн бүх идэвхтэй session-ийг шууд invalid болгож хаана.
- **Хандалтын хязгаарлалт (Rate Limiting):** express-rate-limit middleware-ийг /login болон /verify-otp замууд дээр тавьсан. Үүгээр нэг IP хаягнаас богино хугацаанд олон хандалт хийхийг хязгаарлаж, Brute-force халдлагыг бүрэн зогсоож байгаа юм.

9. Тестийн үр дүн (Test Results)

Бид бичсэн backend кодоо зөв ажиллаж байгааг батлахын тулд **12 төрлийн интеграци тест** бичиж, Jest тестийн орчинд шалгасан. Тестүүд 100% амжилттай давсан.

PASS src/tests/auth.test.js

Хэрэглэгчийн authentication модулийн тестүүд:

- register success (Зөв өгөгдлөөр хэрэглэгч үүсэх)
- duplicate email (Давхардсан имэйлийг reject хийх)
- invalid phone (Буруу утасны дугаарыг validation-оор барих)
- OTP verify success (Зөв OTP оруулж хэрэглэгч идэвхжих)
- OTP expired (Хугацаа нь дууссан OTP-г буцаах)
- OTP 3 failed attempts (3 удаа буруу оруулбал lock хийх)
- login success (Баталгаажсан хэрэглэгч token авах)
- wrong password (Буруу нууц үгийг үл зөвшөөрөх)
- access protected route without token (Token-гүй хандалтыг 401-ээр хаах)
- admin route blocked for non-admin (Жирийн хэрэглэгчийг админ хуудас руу 403-аар хаах)
- refresh token rotates and invalidates old token (Token сэлгэлт амжилттай болох)
- logout invalidates refresh token (Системээс гарахад token-ийг устгах)

10. Үзүүлэн харуулах дараалал (Demo Flow)

Хамгаалалт болон багшид ажлаа танилцуулахдаа дараах практик дарааллын дагуу Postman эсвэл curl ашиглан үзүүлнэ:

1. **Бүртгүүлэх хэсэг:** /register рүү буруу утасны дугаартай өгөгдөл илгээж 400 Validation Failed алдааг харуулна. Дараа нь зөв өгөгдлөөр бүртгүүлнэ.
2. **OTP шалгах:** Бүртгэлийн дараа ирсэн кодоор /verify-otp руу хандаж хэрэглэгчийг идэвхжүүлнэ. Буруу кодыг зориудаар 3 удаа өгч систем 423 Locked болгож байгааг үзүүлнэ.

-
3. **Нэвтрэх ба Эрх шалгах:** Зөв нууц үгээр нэвтэрч `accessToken` авна. Авсан token-оо ашиглан профайл руугаа амжилттай орж үзүүлнэ. Мөн уг жирийн хэрэглэгчийн эрхээр админы `/api/admin/users` рүү хандаж `403 Forbidden` алдаа авч байгааг батална.
 4. **Системээс гарах:** `/logout` хийсний дараа хуучин token-ууд дахин ажиллахгүй идэвхгүй болсныг харуулна.

11. Дүгнэлт

Бие даалтын ажлын хүрээнд манай баг хандивын системийн суурь аюулгүй байдлыг хариуцсан хэрэглэгчийн бүртгэл, эрх зохицуулалтын модулийг амжилттай хөгжүүллээ. Кодыг үелсэн архитектурт оруулж өгснөөр зохиомжийн хувьд цэгцтэй, ойлгомжтой болсон бөгөөд `bcrypt`, `JWT Rotation`, `OTP Attempt Lock` зэрэг тактикуудыг суулгаснаар `brute-force` болон дундаас token замаар нэвтрэх халдлагуудыг backend түвшинд хааж чадсан. Автоматжуулсан интеграци тестүүд бүрэн давсан нь модулийн ажиллагааны найдвартай байдлыг баталж байна.

Төслийн бүх эх кодыг сургуулийн шаардлагын дагуу дараах GitHub репозиторид нээлттэй байршуулсан болно:

URL: <https://github.com/AsilanAsKa/BagiinAjil-8.git>

Ашигласан ном

- [1] Express.js Official Documentation. “Express - Fast, unopinionated, minimalist web framework for Node.js,” <https://expressjs.com/>, [Accessed: April 2026].
- [2] Auth0. “JSON Web Token (JWT) Introduction,” <https://jwt.io/introduction/>, [Accessed: April 2026].
- [3] OWASP Foundation. “OWASP Top 10:2021 - The Most Critical Web Application Security Risks,” <https://owasp.org/www-project-top-ten/>, [Accessed: March 2026].
- [4] Mongoose ODM. “Mongoose v8.0.0 Documentation - Elegant MongoDB Object Modeling for Node.js,” <https://mongoosejs.com/docs/>, [Accessed: April 2026].
- [5] NPM Registry. “bcrypt - Modern password hashing library,” <https://www.npmjs.com/package/bcrypt>, [Accessed: April 2026].
- [6] Jest Core Team. “Jest - Delightful JavaScript Testing Framework,” <https://jestjs.io/>, [Accessed: April 2026].
- [7] Robert C. Martin. *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Prentice Hall, 2017.